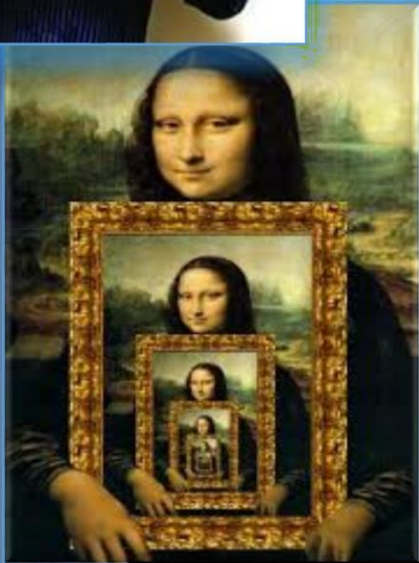


The background is a dark blue digital space. At the top, numerous streams of glowing blue binary code (0s and 1s) fall downwards, creating a sense of data flow. At the bottom, there are several server racks or computer monitors, each displaying a grid of data. A bright blue light source is positioned in the center, casting a glow across the scene.

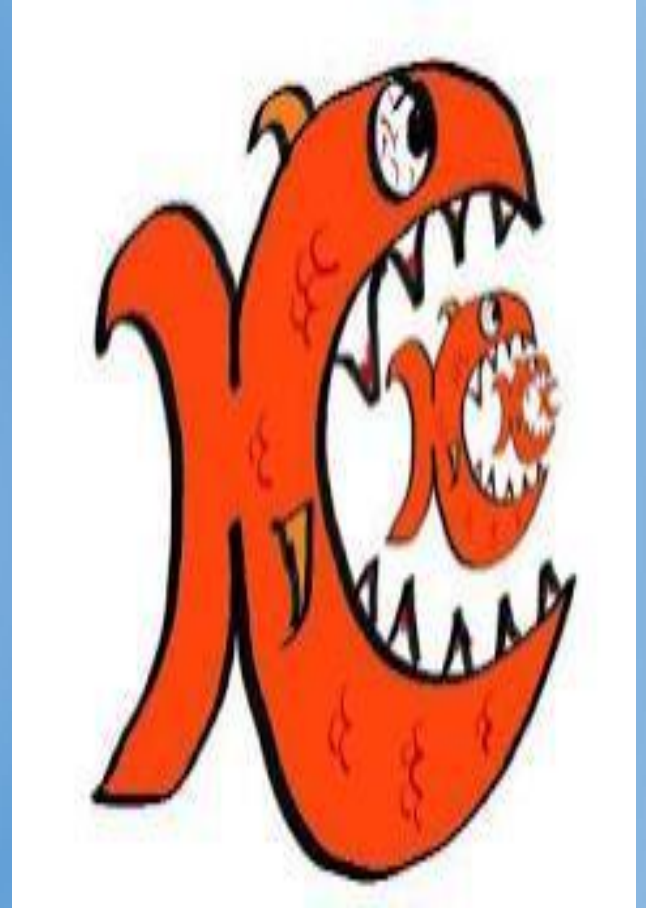
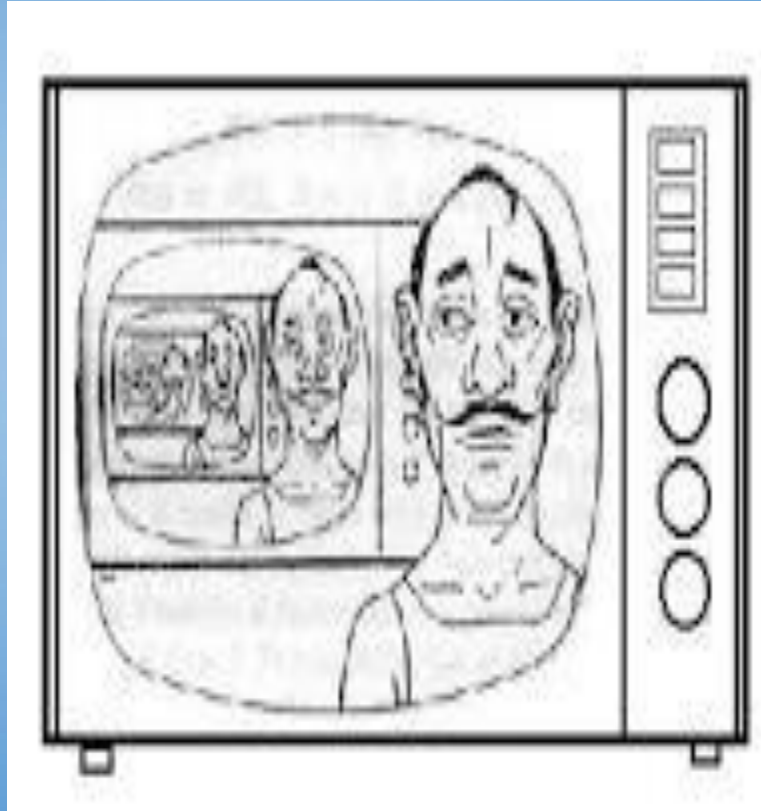
PROGRAMACIÓN I



RECURSIVIDAD



Recursividad-Imágenes



Recursividad

Recursividad es el proceso de definir algo en términos de sí mismo.

- **Se dice que una función es recursiva cuando se llama a sí misma.** El lenguaje C permite que una función pueda llamarse a sí misma. A esta característica se le denomina recursividad.
- **La recursividad es distinto de la iteración**
- **La recursividad es una alternativa para ejecutar estructuras de iteración**

La principal diferencia entre **recursión** e **iteración** radica en su estructura de control: la **recursión** ejecuta un bloque de código llamándose a sí misma de forma repetida hasta cumplir una condición, mientras que la **iteración** utiliza estructuras cíclicas explícitas (como bucles for o while) para repetir un proceso.

Característica	Recursión	Iteración
Definición	Una función se llama a sí misma dentro de su propio cuerpo.	Un bloque de código se repite mediante el uso de bucles explícitos.
Estructura de Control	Utiliza estructuras condicionales (if-else) para detenerse.	Utiliza estructuras cíclicas (for, while, do-while).
Condición de Parada	Definida por el caso base. Si no se alcanza, ocurre un desbordamiento.	Definida por la condición del bucle. Si no se cumple, se genera un bucle infinito.
Uso de Memoria	Alto. Consume memoria de la pila (stack) por cada llamada a la función.	Bajo. Utiliza la misma memoria asignada, modificando variables existentes.

Característica	Recursión	Iteración
Velocidad / Rendimiento	Más lenta. La creación y destrucción de marcos de pila genera sobrecarga.	Más rápida. La ejecución es directa sobre los registros y variables locales.
Legibilidad del Código	Código generalmente más corto, limpio y elegante (estilo declarativo).	Código que puede volverse largo o complejo de leer en problemas lógicos profundos.
Riesgo Crítico	Desbordamiento de la pila (Stack Overflow).	Bucle infinito que congela el programa por consumo de CPU.
Casos de Uso Ideales	Estructuras de datos complejas (árboles, grafos) y algoritmos tipo Divide y Vencerás.	Recorridos simples, operaciones matemáticas repetitivas y arreglos lineales.

Función Recursiva

```
graph LR; A[Función Recursiva] --- B[Caso Base]; A --- C[Caso Recursivo];
```

Caso Base

Solución para un caso particular. Puede haber más de un caso base

Caso Recursivo

Solución que implica usar la función original.

Función Recursiva

Se componen de

1- Caso Base: Una solución simple para un caso en particular (puede haber mas de un caso base).

2- Caso Recursivo: una solución que involucra utilizar la función original, con parámetros que se acercan mas al caso base. Los pasos que sigue el caso recursivo son los siguientes:

- a. El procedimiento se llama a si mismo.
- b. El problema se resuelve, resolviendo el mismo problema pero de tamaño menor.
- c. La manera en la cual el tamaño del problema disminuye asegura que el caso base eventualmente se alcanzará.

Caso: Cálculo del Número Factorial

Escribe un programa que calcule el factorial (!) de un entero no negativo.

Ejemplo:

$$0! = 1$$

$$1! = 1$$

$$2! = 2 \quad \rightarrow \quad 2! = 2 * 1!$$

$$3! = 6 \quad \rightarrow \quad 3! = 3 * 2!$$

$$4! = 24 \quad \rightarrow \quad 4! = 4 * 3!$$

$$5! = 120 \quad \rightarrow \quad 5! = 5 * 4!$$

Función Recursiva: Ejemplo – Factorial

Solución

Aquí se muestra la secuencia que toma el factorial

$$N ! = \begin{cases} 1 & \text{si } N = 0 \text{ (base)} \\ N * (N - 1) ! & \text{si } N > 0 \text{ (recursión)} \end{cases}$$

El razonamiento recursivo tiene dos partes: la **base** y la **regla recursiva de construcción**.

La base NO es recursiva y es el punto de partida como de terminacion de la definicion.

factorial(4)

4 x factorial(3)

return 24

factorial(3)

3 x factorial(2)

return 6

factorial(2)

2 x factorial(1)

return 2

factorial(1)

Caso base

return 1

RESULTADO

24

```
#include <stdio.h>
#include <conio.h>
```

```
long int factorial(int n);
```

```
Int main ()
```

```
{
```

```
    int valor;
```

```
    long int result;
```

```
    printf("\nIngresa numero:");
```

```
    scanf("%d",&valor);
```

```
    if (valor < 0) printf("Error");
```

```
    else
```

```
    {
```

```
        result=factorial(valor);
```

```
        printf("\nEl factorial de %d es %ld",valor,result);
```

```
    }
```

```
    getch();
```

```
    return 0;
```

```
}
```

```
long int factorial(int n)
```

```
{
```

```
    long int resp;
```

```
    if(n==1 || n==0)
```

```
        return 1;
```

```
    else
```

```
        resp=n* factorial(n-1);
```

```
        return (resp);
```

```
}
```

→ **RECURSIVIDAD - factorial
se llama a si misma**

Caso: Sucesión de Fibonacci

La secuencia de Fibonacci fue descubierta por el matemático Leonardo de Pisa, conocido como Fibonacci, en el siglo XIII. Esta secuencia comienza con 0 y 1, y cada número siguiente es la suma de los dos anteriores.

Así: 0, 1, 1, 2, 3, 5, 8, 13, 21, 34... y sigue hasta el infinito.

0 1 1 2 3 5 8 13 21

$$0 + 1 = 1$$

$$1 + 1 = 2$$

$$1 + 2 = 3$$

$$2 + 3 = 5$$

$$3 + 5 = 8$$

$$5 + 8 = 13$$

$$8 + 13 = 21$$

1. Casos base:

- $F(0) = 0$
- $F(1) = 1$

2. Paso recursivo:

- Para hallar $F(n)$, halla recursivamente $F(n-1)$ y $F(n-2)$.
- Luego, calcula $F(n)$ como $F(n-1) + F(n-2)$.

```
int fibonacci(int n) {  
    // Caso base: los dos primeros números de la secuencia  
    if (n == 0) {  
        return 0;  
    } else if (n == 1) {  
        return 1;  
    } else {  
        // Llamada recursiva  
        return fibonacci(n - 1) + fibonacci(n - 2);  
    }  
}
```

Caso: POTENCIA

$$a^0 = 1$$

$$a^1 = a$$

$$a^2 = a \cdot a$$

$$a^3 = a \cdot a \cdot a = a^2 \cdot a$$

$$a^4 = a \cdot a \cdot a \cdot a = a^3 \cdot a$$

$$a^5 = a \cdot a \cdot a \cdot a \cdot a = a^4 \cdot a$$

.....

$$a^n = a \cdot a \cdot a \cdot a \dots \dots \dots a = a^{n-1} \cdot a$$

Caso: POTENCIA

$$a^0 = 1$$

$$a^1 = a$$

$$a^2 = a \cdot a$$

$$a^3 = a \cdot a \cdot a = a^2 \cdot a$$

$$a^4 = a \cdot a \cdot a \cdot a = a^3 \cdot a$$

$$a^5 = a \cdot a \cdot a \cdot a \cdot a = a^4 \cdot a$$

.....

$$a^n = a \cdot a \cdot a \cdot a \dots \dots \dots a = a^{n-1} \cdot a$$

1. Casos base:

- $F(0) = 0$
- $F(1) = 1$

2. Paso recursivo:

- Para hallar $F(n)$, halla recursivamente $F(a, n-1)$ por a .
- Luego, calcula $F(n)$ como $F(a, n-1) * a$.

Actividad

Realizar un programa recursivo que calcule la potencia de un numero entero.

¿Cómo se puede detectar soluciones que podrían resolverse de forma más fácil con una aproximación recursiva?



Cuando se tiene un problema que tiene varios pasos, y el resultado de ejecutar cada paso, deja un problema igual al anterior.

Comparación Iteración - Recursión

- Tanto la iteración como la recursión se basan en una estructura de control: la iteración utiliza una estructura de repetición, la recursión utiliza una estructura de selección.
- Tanto la iteración como la recursión implican repetición: la iteración utiliza la estructura de repetición en forma explícita, la recursión consigue la repetición mediante llamadas de función repetidas.
- La iteración y la recursión ambas involucran una prueba de terminación: la iteración termina cuando falla la condición de continuación del ciclo, la recursión termina cuando se reconoce un caso base. La recursión tiene muchas negativas.
- La sobrecarga de llamadas de la función puede resultar costoso tanto en tiempo de procesador como en espacio de memoria.
- Cada llamada recursiva genera otra copia de la función, esto puede consumir gran cantidad de memoria.
- La iteración por lo regular ocurre dentro de una función por lo que no ocurre la sobrecarga de llamadas repetidas de función y asignación extra de memoria.

Bibliografía

<https://academiasanroque.com/60-ejercicios-de-funciones-recursivas-en-python-para-la-carrera-de-informatica/>

<https://cyminversores.com/secuencia-de-fibonacci-hasta-en-el-trading/>